

Controle de Estoque e Vendas

Documentação de testes da API

Node.js, Express, MongoDB e JWT

Alunos: Sidney Alves, Lavinia Dias

Trabalho de Tópicos em Sistemas da Informação - UEMS

Sobre este documento

Este documento reúne os testes feitos na API depois que ela ficou pronta, pra comprovar que as funcionalidades pedidas no trabalho estão funcionando: autenticação com JWT, senha criptografada com bcrypt, CRUD das entidades, controle de estoque nas vendas e o tratamento de erros e permissões.

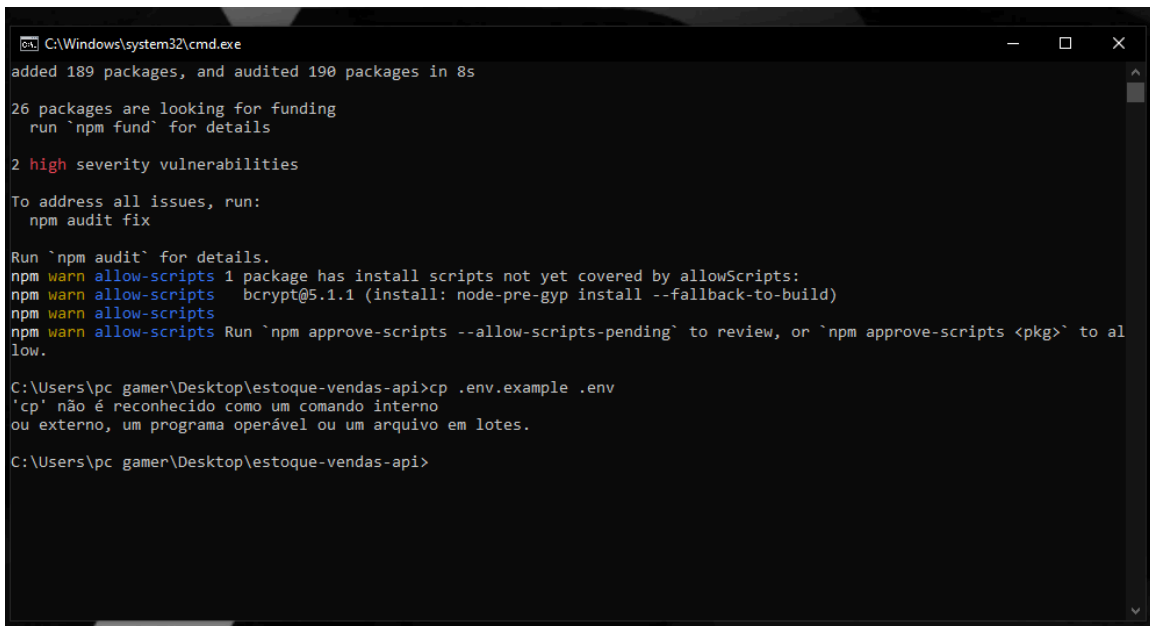
Os testes foram feitos com o Insomnia, usando a coleção de requisições que está na raiz do repositório. O banco usado foi um cluster gratuito do MongoDB Atlas, e o servidor rodou localmente na máquina.

Ambiente

Runtime	Node.js
Banco de dados	MongoDB Atlas (cluster M0)
Cliente HTTP usado nos testes	Insomnia
Sistema operacional	Windows

1. Preparando o ambiente

As dependências do projeto foram instaladas com **npm install** e, depois de configurar o arquivo `.env` com a string de conexão do MongoDB Atlas, o servidor foi iniciado com **npm run dev**.

A terminal window titled 'C:\Windows\system32\cmd.exe' showing the output of an 'npm install' command. The output includes information about installed packages (189), audited packages (190), and funding. It also shows two high severity vulnerabilities and a warning about 'allow-scripts' in the 'bcrypt@5.1.1' package. The user then attempts to copy a file from the project directory but receives an error: 'cp' is not recognized as an internal or external command, program operable or a batch file.

```
C:\Windows\system32\cmd.exe
added 189 packages, and audited 190 packages in 8s

26 packages are looking for funding
  run `npm fund` for details

2 high severity vulnerabilities

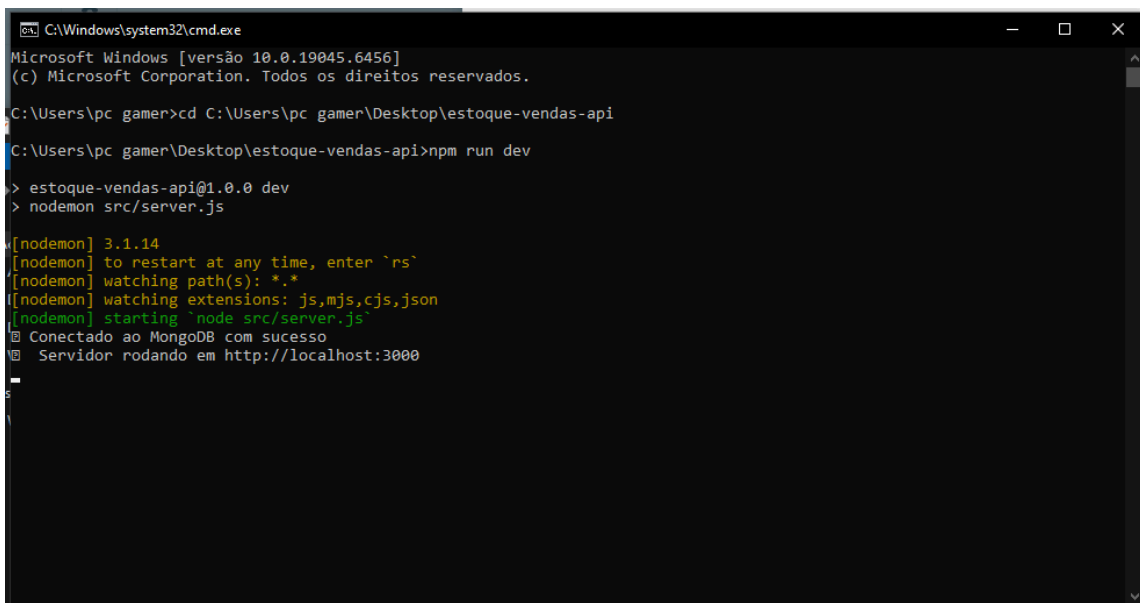
To address all issues, run:
  npm audit fix

Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   bcrypt@5.1.1 (install: node-pre-gyp install --fallback-to-build)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to al
low.

C:\Users\pc gamer\Desktop\estoque-vendas-api>cp .env.example .env
'cp' não é reconhecido como um comando interno
ou externo, um programa operável ou um arquivo em lotes.

C:\Users\pc gamer\Desktop\estoque-vendas-api>
```

npm install rodando na pasta do projeto.

A terminal window titled 'C:\Windows\system32\cmd.exe' showing the output of an 'npm run dev' command. The user navigates to the project directory and runs the command. The output shows the version of the project (estoque-vendas-api@1.0.0) and the dev script (nodemon src/server.js). It then shows the nodemon version (3.1.14) and its configuration (watching path(s): *.* and extensions: js,mjs,cjs,json). The terminal also shows messages indicating that the server is connected to MongoDB successfully and is running on http://localhost:3000.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [versão 10.0.19045.6456]
(c) Microsoft Corporation. Todos os direitos reservados.

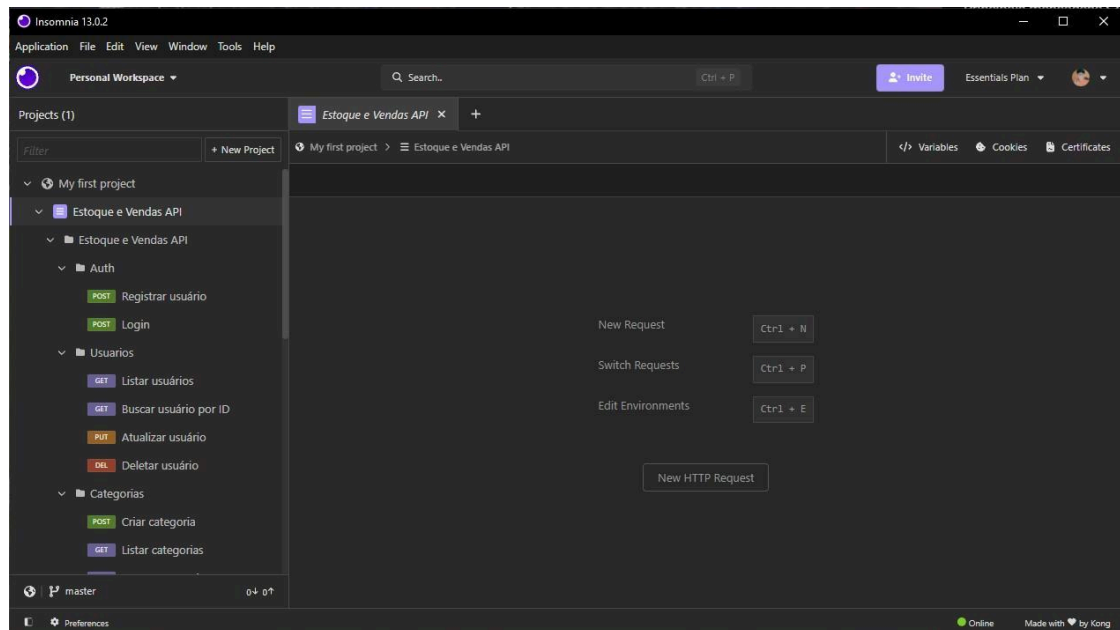
C:\Users\pc gamer>cd C:\Users\pc gamer\Desktop\estoque-vendas-api
C:\Users\pc gamer\Desktop\estoque-vendas-api>npm run dev

> estoque-vendas-api@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
Conectado ao MongoDB com sucesso
Servidor rodando em http://localhost:3000
```

Servidor no ar e conectado ao banco — mensagens "Conectado ao MongoDB com sucesso" e "Servidor rodando em http://localhost:3000".

A coleção de requisições foi importada no Insomnia (arquivo .json que fica na raiz do repositório), o que já organiza as rotas em pastas por entidade: Auth, Usuarios, Categorias, Fornecedores, Produtos e Vendas.

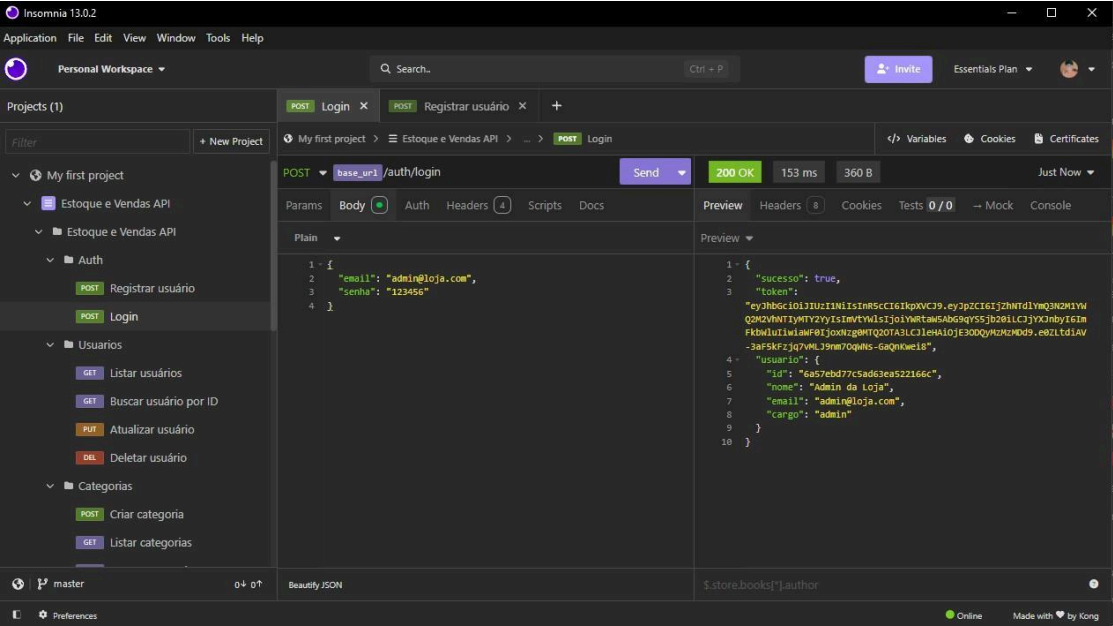


Coleção importada no Insomnia.

2. Testes realizados

2.1 Autenticação

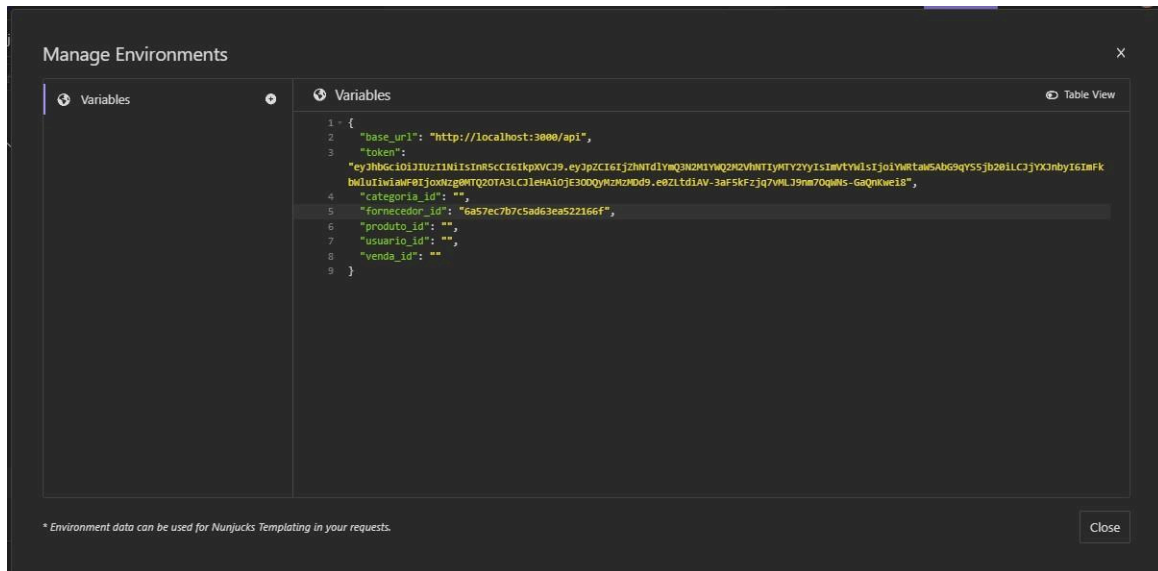
Primeiro foi criado um usuário administrador (POST /api/auth/registrar) e depois feito o login (POST /api/auth/login). A senha enviada não fica salva em texto plano — o Model faz o hash com bcrypt antes de gravar no banco. O login, quando os dados batem, devolve um token JWT que passou a ser usado nas requisições seguintes.



Login feito com sucesso, token JWT retornado junto com os dados do usuário.

2.2 Cadastro de categoria, fornecedor e produto

Com o token do admin nas variáveis da coleção, foram cadastrados uma categoria, um fornecedor e um produto associado aos dois — testando o CRUD dessas entidades e os relacionamentos entre elas (categoria e fornecedor são referenciados no produto).

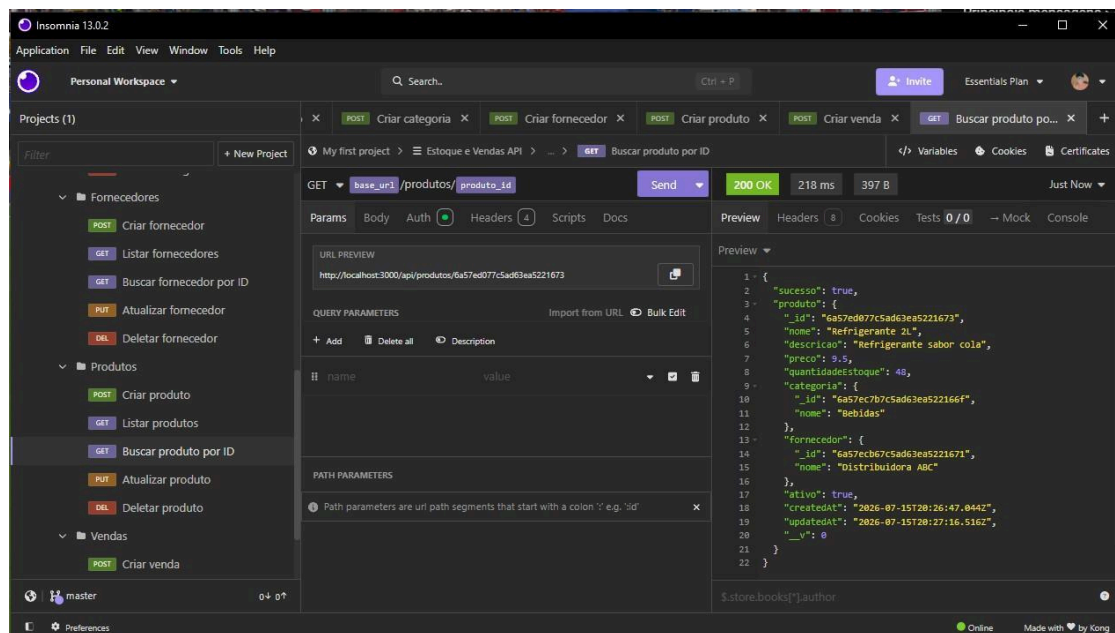


IDs gerados após a criação de categoria, fornecedor e produto, salvos nas variáveis do Insomnia.

2.3 Venda e baixa de estoque

Esse foi o teste mais importante, porque é a regra de negócio central do sistema: ao registrar uma venda (POST /api/vendas), a API confere se tem estoque suficiente, calcula o total da venda sozinha e desconta a quantidade vendida do produto.

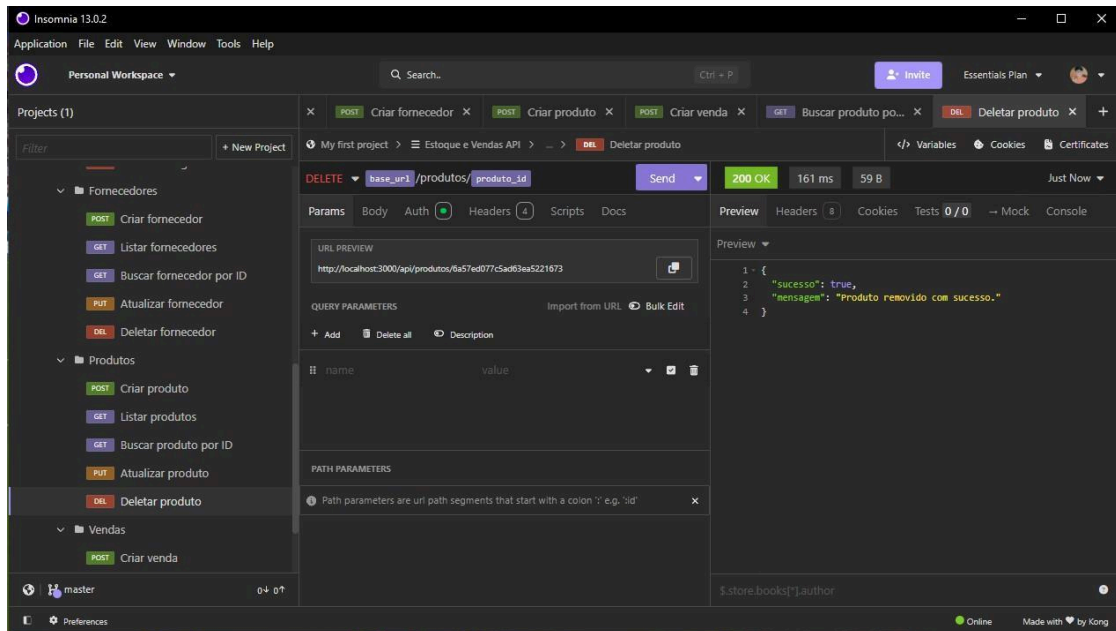
O produto tinha sido cadastrado com 50 unidades em estoque. Depois de uma venda de 2 unidades, consultando o produto de novo o estoque aparece com 48 — confirmando que a baixa aconteceu certo.



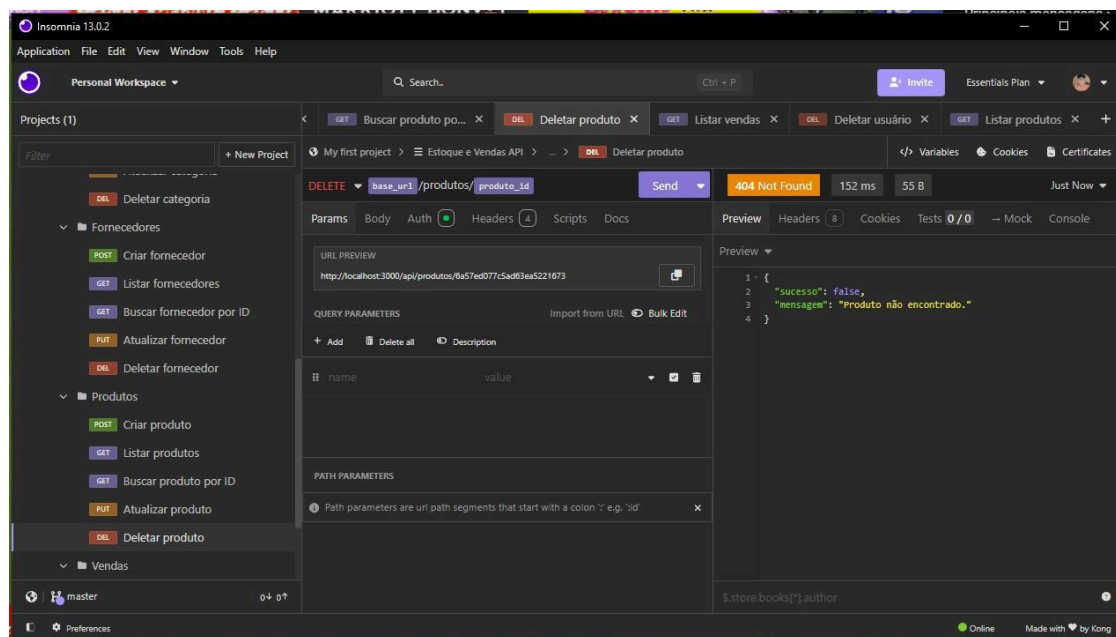
quantidadeEstoque = 48 depois da venda de 2 unidades de um produto que tinha 50.

2.4 Exclusão de produto e erro 404

Pra fechar o CRUD, o produto foi removido com DELETE /api/produtos/:id. Em seguida, tentei excluir o mesmo produto de novo, só pra ver como a API se comporta quando o ID não existe mais no banco — o servidor não trava, ele retorna 404 com uma mensagem explicando o erro.



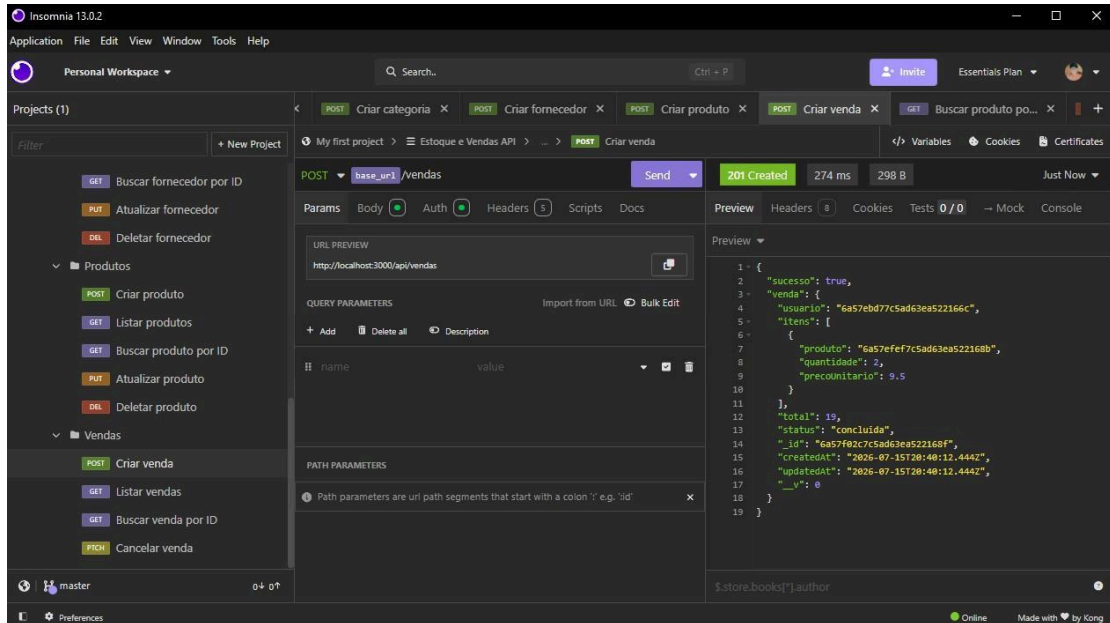
Produto removido com sucesso (200 OK).



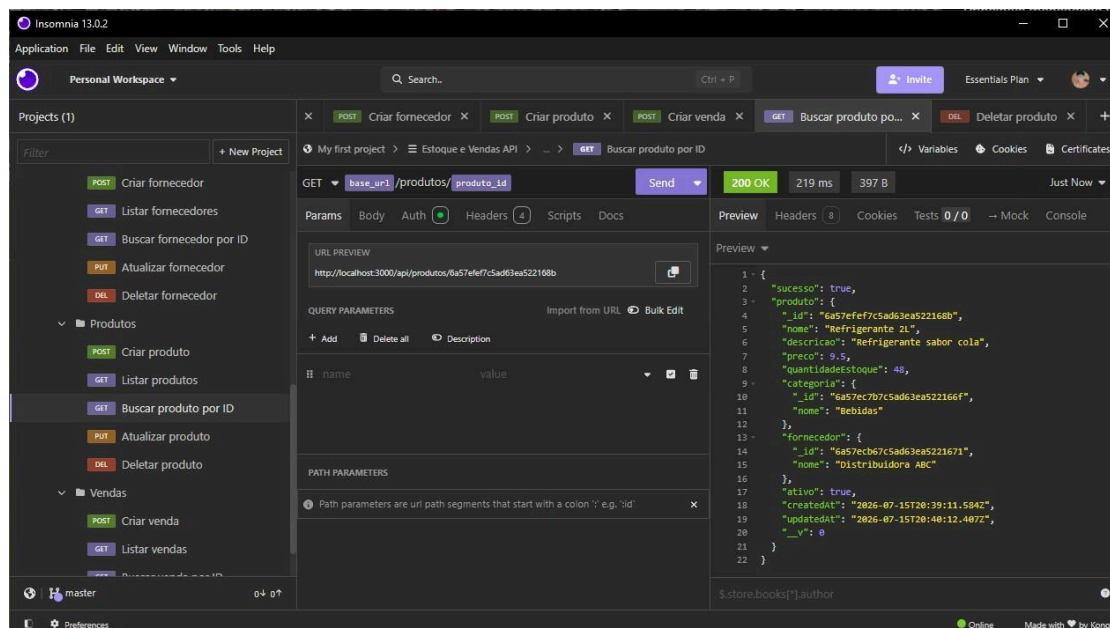
Tentativa de excluir o mesmo produto de novo: 404 Not Found, "Produto não encontrado."

2.5 Cancelamento de venda

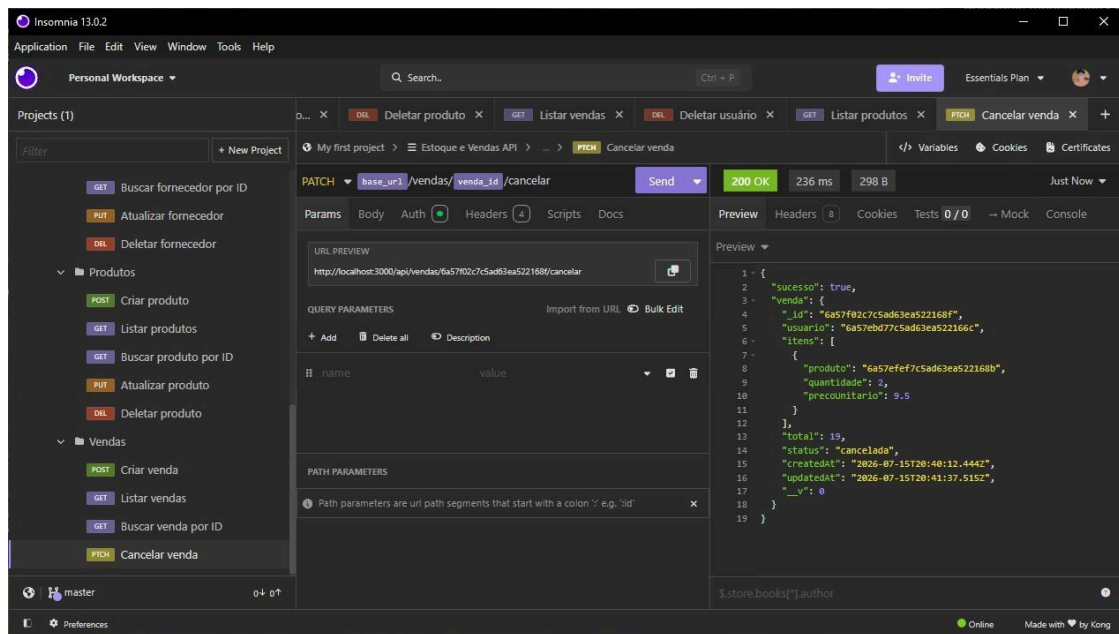
Como o produto usado na venda anterior tinha sido apagado, criei um produto novo e registrei outra venda em cima dele, pra testar o cancelamento (PATCH /api/vendas/:id/cancelar). A ideia aqui é que cancelar uma venda devolva as unidades pro estoque, em vez de simplesmente apagar o registro — assim o histórico não se perde.



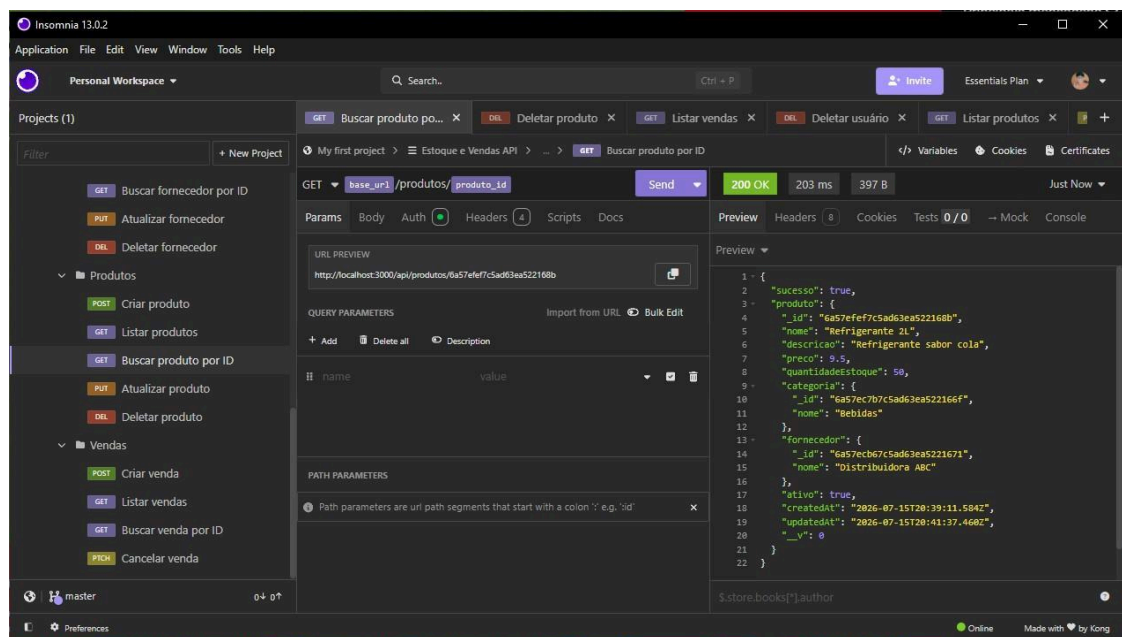
Nova venda registrada (201 Created), total calculado automaticamente.



Estoque em 48 depois da venda, antes do cancelamento.



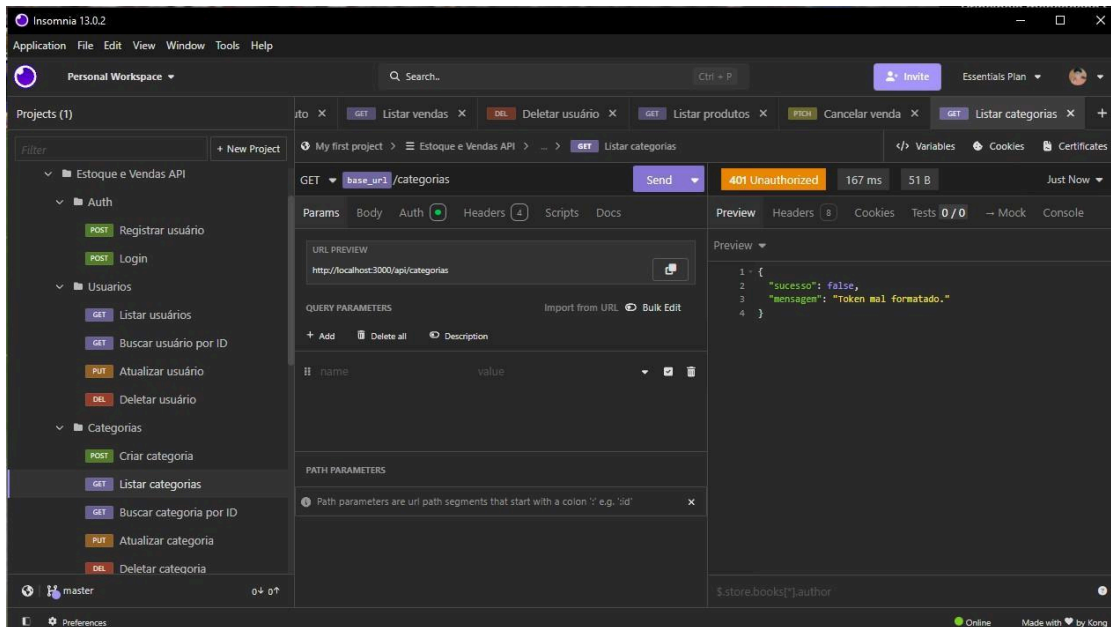
Venda cancelada — status alterado para "cancelada" (200 OK).



Estoque de volta a 50 unidades depois do cancelamento — a devolução funcionou.

2.6 Rota protegida sem token (erro 401)

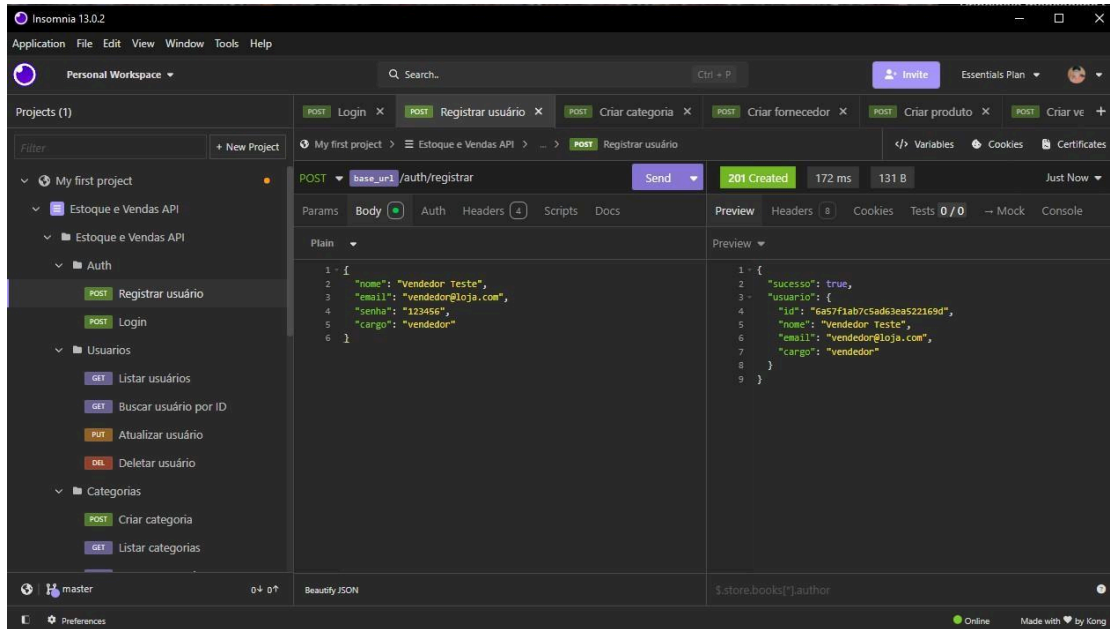
Removi o token das variáveis da coleção e tentei acessar uma rota protegida (GET /api/categorias) sem autenticação. O middleware de autenticação barrou o acesso antes mesmo de chegar no controller.



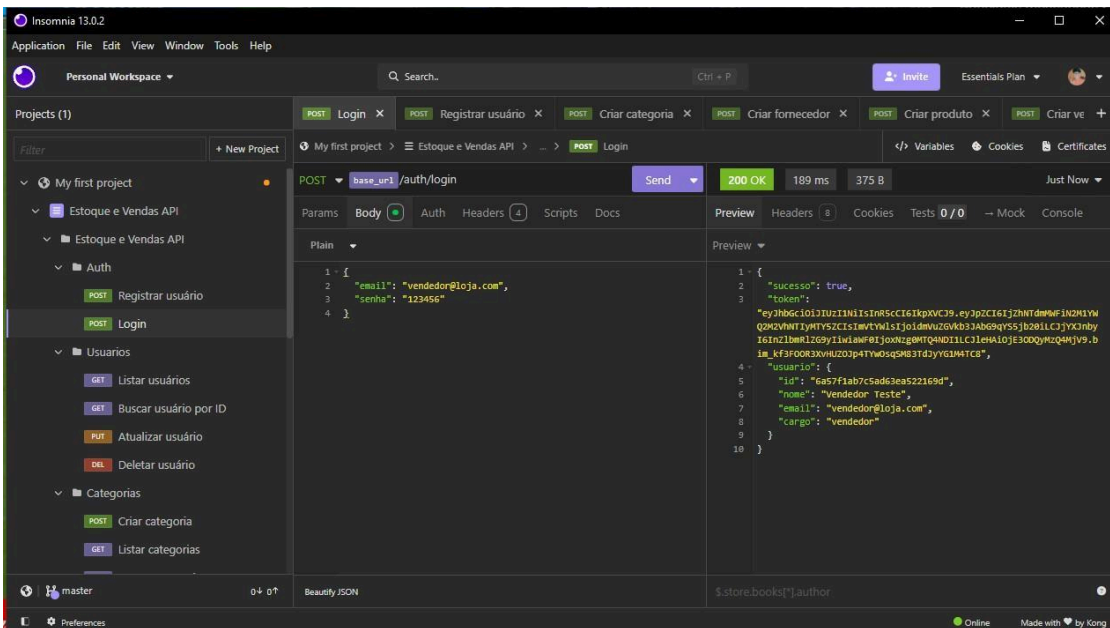
401 Unauthorized ao tentar acessar sem token.

2.7 Permissão por cargo (erro 403)

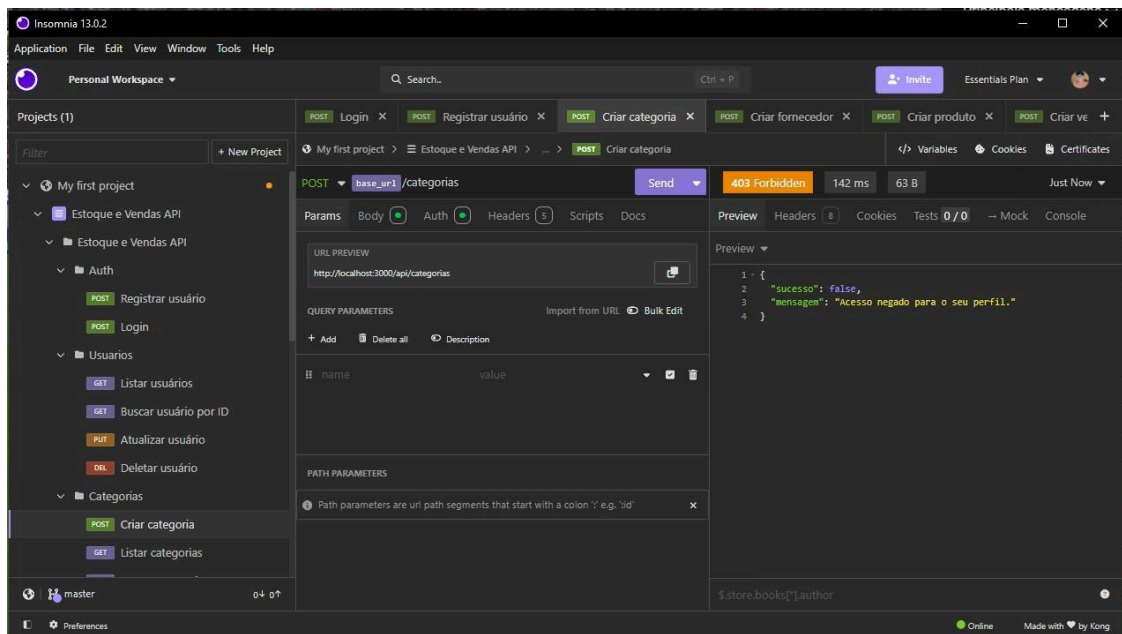
Pra testar o controle de acesso por perfil, criei um segundo usuário com cargo "vendedor" — que na regra do projeto pode consultar o catálogo e registrar vendas, mas não pode alterar cadastros. Depois de logar com esse usuário, tentei criar uma categoria, ação restrita a administradores.



Usuário vendedor registrado.



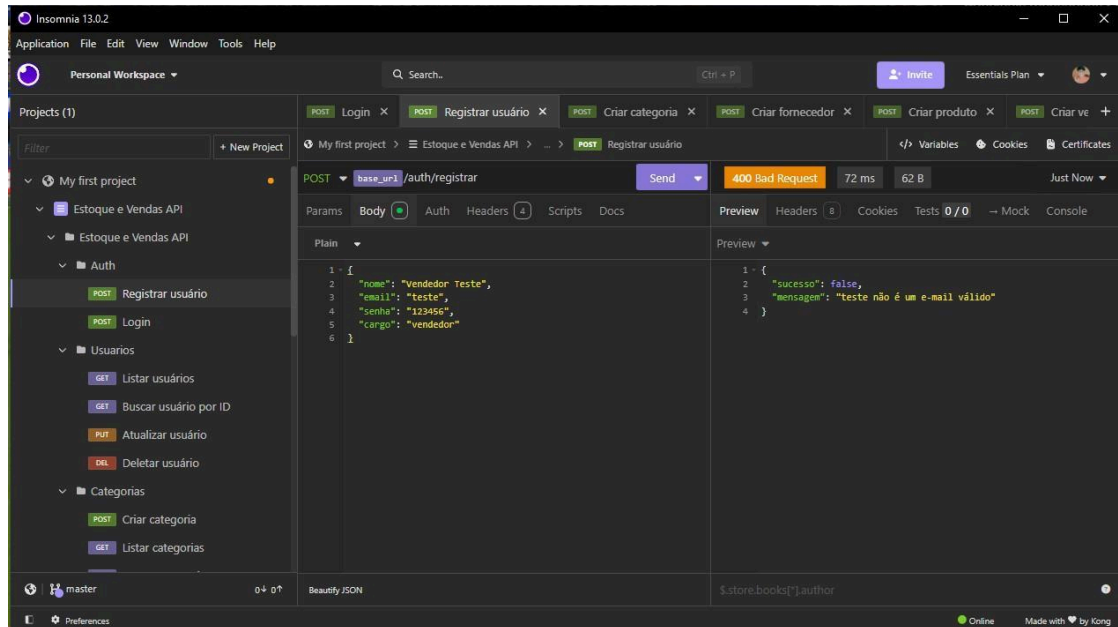
Login do vendedor, token gerado com cargo "vendedor".



Tentativa de criar categoria logado como vendedor: 403 Forbidden, "Acesso negado para o seu perfil."

2.8 Validação de e-mail (erro 400)

Por último, testei a validação por regex no Model de usuário, tentando registrar um usuário com um e-mail sem formato válido ("teste", sem @ nem domínio).



400 Bad Request — "teste não é um e-mail válido".

3. Resumo

A tabela abaixo reúne os testes feitos e o status HTTP que cada um retornou.

Teste	Rota	Status
Registro de usuário	POST /api/auth/registrar	201
Login	POST /api/auth/login	200
Criar categoria / fornecedor / produto	POST /api/categorias, /fornecedores, /produtos	201
Registrar venda (baixa de estoque)	POST /api/vendas	201
Excluir produto	DELETE /api/produtos/:id	200
Buscar produto já excluído	DELETE /api/produtos/:id	404
Cancelar venda (devolução ao estoque)	PATCH /api/vendas/:id/cancelar	200
Acesso sem token	GET /api/categorias	401
Ação sem permissão de cargo	POST /api/categorias	403
E-mail inválido no registro	POST /api/auth/registrar	400

Com esses testes, ficam cobertos os pontos principais pedidos no trabalho: autenticação com JWT, senha protegida com bcrypt, CRUD das entidades, a regra de baixa e devolução de estoque nas vendas, e o tratamento de erros (400, 401, 403 e 404) sem que o servidor caia em nenhum momento.